QuestChain

QuestChain Whitepaper

Version 1.0 | BOT Chain

Trustless Quest Infrastructure for On-Chain Campaigns

Network	**BOT Chain**
Chain ID	**6805**
Contract	**QuestChain.sol (MIT)**
Platform Fee	**2% (200 bps)**

ABSTRACT

QuestChain is a decentralised quest-creation and participation platform deployed on BOT Chain. It allows any wallet to permissionlessly set up quests — structured on-chain tasks paired with native-token reward pools — and for any participant to submit cryptographically timestamped proof of completion. Reward distribution is creator-controlled and fully on-chain, eliminating intermediaries, payment delays, and trust assumptions that characterise existing Web2 campaign tools.

1. Problem

Growth campaigns and on-chain quests are a core acquisition mechanism for crypto projects. Yet the dominant tooling (Galxe, Layer3, Questn, Zealy) shares structural weaknesses:

Pain Point	Description
Custody risk	Reward pools sit in platform-controlled multisigs. Rug risk is non-zero.
Off-chain verification	Proof of completion is centralised; subjective decisions have no audit trail.
Delayed payouts	Manual review cycles create payout delays of days to weeks.
Platform dependency	Projects lose campaign history and participant data if a platform shuts down.
Fee opacity	Pricing is opaque, often 5-15% of reward pools plus subscription costs.

2. Solution

QuestChain moves the entire quest lifecycle on-chain:

- **Pool creation** — Creator deploys reward pool directly into the smart contract.
- **Proof submission** — Participants submit proof (IPFS CID, tx hash, URL) immutably on-chain.
- **Review & payout** — Creator reviews and approves; reward transfers atomically to the winner's wallet.
- **Refund logic** — Unclaimed pools after deadline are returnable to creators. Cancelled quests refund immediately.

Nothing is off-chain except the content of the proof itself, which is referenced by an IPFS CID pinned independently.

3. Architecture

3.1 Smart Contract — QuestChain.sol

Deployed as a single non-upgradeable contract on BOT Chain. Core data structures:

```
Quest {
  id, creator, title, descriptionCID,
  rewardPerWinner, maxWinners, deadline,
  status (Active | Paused | Completed | Cancelled),
  winnersCount, totalRewardPool
}

Submission {
  id, questId, participant, proofCID,
  status (Pending | Approved | Rejected),
  timestamp
}
```

Key invariants enforced by contract:

- One submission per participant per quest.
- Reward release only on explicit creator approval.
- Pool cannot exceed rewardPerWinner x maxWinners; excess msg.value is refunded atomically.
- After deadline, creator may reclaim unfilled pool.
- Quest cancellation refunds remaining pool minus already-approved payouts.

3.2 Platform Fee

A configurable basis-points fee (default: 200 bps / 2%) is charged at quest creation time on the total reward pool. Fees accumulate in the contract and are withdrawable only by the contract owner. The fee is transparent and known before transaction confirmation. Maximum fee is capped at 1000 bps (10%) and can only be lowered or maintained, never raised above the cap.

3.3 Frontend

A single-page application (index.html) with zero backend dependency. It communicates directly with the deployed contract via ethers.js (v6, loaded from CDN). Pages:

- **Explore** — Browse all quests with live status and reward data pulled from chain.
- **My Quests** — Creator dashboard for managing quests and reviewing submissions.
- **Create Quest** — Form to configure and deploy a quest with cost preview.
- **My Submissions** — Participant view of all historical submissions and statuses.
- **Quest Detail** — Full quest view with proof submission, creator controls, and submission review table.

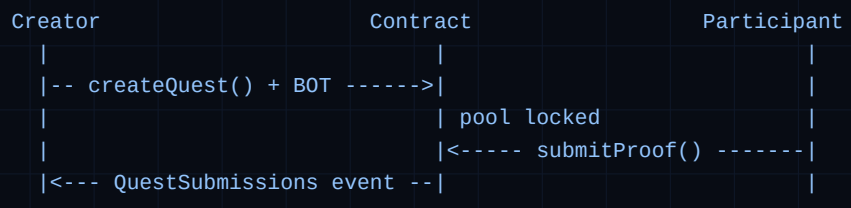
3.4 Network

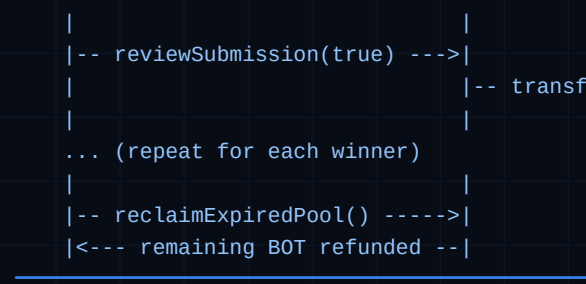
Property	Value
Network	BOT Chain (Botanix Labs)
Chain ID	6805
Native Token	BOT
RPC	https://rpc.botanixlabs.dev
Explorer	https://blockscout.botanixlabs.dev

BOT Chain is a Bitcoin-secured EVM-compatible L2, making it appropriate for value-bearing contracts like reward pools. Its EVM compatibility means the contract deploys with standard Foundry / Hardhat tooling.

---

4. Quest Lifecycle





5. Security Considerations

5.1 Reentrancy

All state mutations (winnersCount, submission status) are written before any transfer() call, following the checks-effects-interactions pattern. No .call{value}() with arbitrary gas is used.

5.2 Access Control

Creator-only functions are enforced by onlyQuestCreator(questId) which reads quests[id].creator. Owner-only functions (fee withdrawal, fee config) use onlyOwner. No role delegation or proxy patterns; access control is deterministic.

5.3 Duplicate Submission Prevention

participantSubmission[questId][address] maps each participant to exactly one submission ID per quest. Any second submitProof call reverts with 'Already submitted'.

5.4 Reward Drain

The contract holds exactly rewardPerWinner x maxWinners for each quest. It is impossible to approve more winners than maxWinners; the check q.winnersCount < q.maxWinners is re-evaluated on every reviewSubmission call.

5.5 Griefing / Spam

Submission slots can be theoretically filled with worthless proofs. Mitigations for future versions include staking a small deposit per submission (returned on approval, slashed on repeated rejection).

5.6 Centralised Review

Proof review is centralised to the quest creator — a deliberate design choice mirroring how campaign managers work. Future versions can introduce a DAO-based arbiter for disputed reviews.

6. Tokenomics

QuestChain itself does not issue a token. BOT (the native gas and reward token of BOT Chain) is the sole settlement currency in v1. This avoids liquidity bootstrapping complexity and aligns with BOT Chain's native token economy.

Flow	Direction
Creator → Contract	rewardPool + platformFee at creation
Contract → Winner	rewardPerWinner on each approval
Contract → Creator	Remaining pool on cancel or reclaim
Contract → Owner	Accumulated platform fees on withdrawFees()

7. Deployment

Prerequisites

- Foundry (forge, cast)
- BOT Chain RPC and a funded deployer wallet

Deploy Command

```
forge create src/QuestChain.sol:QuestChain \
  --constructor-args 200 \
  --rpc-url https://rpc.botanixlabs.dev \
  --private-key $PRIVATE_KEY \
  --verify \
  --verifier blockscout \
  --verifier-url https://blockscout.botanixlabs.dev/api
```

Post-deploy Steps

1. Copy the deployed contract address.
2. Replace CONTRACT_ADDRESS in index.html.
3. Host index.html on GitHub Pages, Vercel, or IPFS.

8. Roadmap

Phase	Target	Scope
v1 — Foundation	Now	Quest creation, proof submission, manual review, refund logic, frontend
v2 — Verification	Q1 2025	On-chain oracles for common tasks (follow on X, hold NFT, make tx)
v3 — Reputation	Q2 2025	Participant reputation scores; submission stake/slash to deter spam
v4 — Governance	Q3 2025	Arbiter committee for disputed reviews; DAO-controlled fee parameters
v5 — Multi-chain	Q4 2025	Quest campaigns spanning multiple EVM chains via cross-chain messaging

9. Comparison

Feature	QuestChain	Galxe	Layer3	Zealy
Fully on-chain	YES	NO	NO	NO
Non-custodial pool	YES	NO	NO	NO
Permissionless creation	YES	Partial	Partial	NO
Atomic payout	YES	NO	NO	NO
Open proof record	YES	NO	NO	NO
Platform subscription	None	Yes	Yes	Yes
Fee transparency	2% on-chain	Opaque	Opaque	Opaque

10. License

QuestChain.sol is released under the MIT License.

Appendix A — Contract Interface Summary

```
// Quest management
function createQuest(string title, string descriptionCID,
    uint256 rewardPerWinner, uint256 maxWinners,
    uint256 deadline) external payable returns (uint256 questId);
function togglePause(uint256 questId) external;
function cancelQuest(uint256 questId) external;
function reclaimExpiredPool(uint256 questId) external;

// Participation
function submitProof(uint256 questId, string proofCID) external;

// Review
function reviewSubmission(uint256 submissionId, bool approve) external;

// Views
function quests(uint256) external view returns (Quest memory);
function submissions(uint256) external view returns (Submission memory);
function getQuestSubmissions(uint256) external view returns (uint256[] memory);
function getSubmissionForParticipant(uint256, address)
    external view returns (uint256);

// Admin
function withdrawFees() external;
function setFeeBps(uint256 newFeeBps) external;
function transferOwnership(address newOwner) external;
```

QuestChain — built on BOT Chain | botchain.ai | scan.botchain.ai